ENGINEERING NOTEBOOK

Accord

REST + GraphQL, and which API test catches which breaking change

Author Nikhil Sinha

Repository github.com/Nikhil201716/Accord-Contract-Testing

Stack Python · FastAPI · Strawberry GraphQL · Pact-style contracts · pytest + Hypothesis · Newman

Edition 1.0

How to read this book

This is an API-testing notebook that happens to be about a book catalog. It is written for someone building or testing services who wants to know, concretely, which testing mechanism catches which kind of breaking change -- and which kinds escape them all.

Every listing is extracted from the repository at build time and every number is read from the reports a real run wrote, so nothing here can drift from the code.

Contents

1. No single API test catches everything

1.1 The finding

1.2 Why a book catalog over REST and GraphQL

2. One catalog, two surfaces

2.1 The data

2.2 One app, two routers

3. The store, queried

3.1 Shared queries

3.2 Lookup by author

4. The REST surface

4.1 The response shape

5. The rest of the REST

5.1 The declared shapes

5.2 List, create, by author

5.3 The by-author endpoint

6. Status codes as a contract

6.1 The vocabulary

7. Testing the unhappy paths

7.1 Errors are part of the API

8. The GraphQL surface

8.1 The schema

9. GraphQL introspection and the type map

9.1 Building the type map

10. GraphQL's different error model

10.1 Null, and the errors array

11. The provider's own tests

11.1 What the provider tests

11.2 Behaviour, thoroughly

11.3 Listing the GraphQL catalog

12. The seed, and testing against known data

12.1 Known data, exact assertions

13. The consumer and its contract

13.1 The consumer

13.2 The contract

13.3 The GraphQL side of the contract

14. The consumer, over GraphQL

14.1 One client, two protocols

15. Consumer-driven contract verification

15.1 Verifying the provider against the consumer's needs

16. Matching types across two protocols

16.1 One matcher, both surfaces

17. The schema diff

17.1 Derive a descriptor, diff it

18. The change catalog

18.1 A change is one or more edits

18.2 Two more entries

18.3 The type-change entries

19. The scoring harness

19.1 Apply and measure

20. Running a mechanism against patched code

20.1 A fresh interpreter per probe

21. Turning the matrix into the finding

21.1 Complementarity, computed

22. Results

22.1 The catch matrix

22.2 Who catches what the others miss

22.3 The gaps

23. Tracing four changes

23.1 C01 rename: contract catches, provider misses

23.2 C15 nullability: only schema-diff catches

23.3 C12 semantic: only the value matcher catches

23.4 C14 semantic format: nothing catches

24. Consumer tests, and the false comfort of mocks

24.1 The consumer against the real provider

25. REST versus GraphQL, for the tester

25.1 Typed schema, cheaper diff

26. The performance tests GraphQL needs

26.1 N+1 and query cost

27. Avoiding the break in the first place

27.1 Expand, migrate, contract

27.2 Deprecation is a first-class API feature

28. Contract-first, schema-first

28.1 Design the promise, then keep it

29. Contracts at scale

29.1 The multi-repo version

30. Precision: not flagging the safe changes

30.1 Additive changes must pass silently

31. Security testing for a read API

31.1 What matters when the data is public

32. Design decisions

32.1 Why a curated catalog

32.2 Why value matchers in the contract

32.3 Why subprocess probes

33. The API testing pyramid

33.1 From free to expensive

34. The test tooling

34.1 Configuration as code

35. Input validation, and the gap it leaves

35.1 Rejecting bad writes

36. Fuzzing the write path

36.1 No unhandled crash

37. Every mechanism in CI

37.1 The pipeline

38. From tests to production monitoring

38.1 A contract is a synthetic check

39. Reproducibility

39.1 Re-run and diff

40. The finding, restated

40.1 Everything, in one place

40.2 The paragraph

41. What this does not establish

41.1 A curated catalog, one consumer

41.2 'Caught' means one check failed

42. Where to read further

42.1 The sources

43. Running the Project, Step by Step

43.1 Prerequisites

43.2 The provider, tests, and the finding

43.3 Explore the API by hand

43.4 Postman / Newman (optional)

43.5 What you should see

44. Appendix

44.1 A - Repository map

44.2 B - Glossary

CHAPTER 1

No single API test catches everything

> *REST and GraphQL, three testing mechanisms, and a catalog of breaking changes -- measured. Each mechanism owns a different class of break, and some breaks escape them all.*

1.1 The finding

Accord is a book-catalog API served over **both REST and GraphQL**, with a reading-list consumer and a Pact-style contract. A harness injects a catalog of API changes one at a time and scores three testing mechanisms against each: the provider's own tests, consumer-driven contract verification, and a structural schema diff.

Of 13 breaking changes, provider tests catch 5, consumer contracts catch 7, schema diff catches 10; 2 slip past all three. Contracts catch 5 that the provider's own tests miss. No mechanism flagged any of the 2 additive changes (0 false positives).

Table 1.1 - catch counts by mechanism (reports/summary.json)

Mechanism	Breaking caught (of 13)	What it uniquely owns
provider's own tests	5	changes that break what the provider chose to assert
consumer-driven contract	7	what the consumer depends on -- incl. breaks the provider never tested
schema diff (structural)	10	structural / type breaks, with no consumer needed

Of 13 breaking changes, 11 are caught by at least one mechanism and 2 slip past all three. Crucially the three are *complementary, not ranked*: contracts catch 5 changes the provider's own tests miss, schema diff catches 4 the contract is blind to, and the contract catches 1 the schema diff cannot see. There is no one tool to buy.

1.2 Why a book catalog over REST and GraphQL

The domain is small on purpose; the point is the testing. Serving the same data two ways -- REST's untyped JSON and GraphQL's typed, introspectable schema -- lets the study measure something real: a typed schema turns a whole class of breaking change into a structural diff, while REST needs the shape derived from live responses. Both are scored side by side.

- ['**provider/** - FastAPI REST + a Strawberry GraphQL schema over one store.', '**consumer/** - a reading-list client and its Pact-style contract.', "**testing/** - the provider's own tests and consumer integration tests.", '**scoring/** - the contract verifier, the schema diff, the harness, a fuzzer.', '**reports/** - the source of every number here.']

CHAPTER 2

One catalog, two surfaces

> *The store, and the single FastAPI app that exposes it as REST and GraphQL.*

2.1 The data

A book is a plain dataclass; the store is a fixed, seeded set of them, so every test and every scored run sees the same catalog and the results reproduce.

Listing 2.1 - the Book record — provider/app/store.py

```
class Book:
    id: int
    title: str
    author: str
    isbn: str
    year: int                       # year of publication
    tags: list[str] = field(default_factory=list)
    available: bool = True
    cover_url: str = ""
    rating: float = 0.0
```

2.2 One app, two routers

The app factory mounts a REST router and a GraphQL router over the same store, so the two surfaces are always in sync on the data and differ only in how they describe it.

Listing 2.2 - REST + GraphQL from one factory — provider/app/main.py

```
def create_app() -> FastAPI:
    app = FastAPI(title="Accord", version="1.0")
    store = Store()
    app.state.store = store

    app.include_router(build_rest_router(store))

    schema = build_schema(store)
    app.state.schema = schema
    app.include_router(GraphQLRouter(schema), prefix="/graphql")
```

CHAPTER 3

The store, queried

> *The query surface both APIs share -- filtering, lookup, creation.*

3.1 Shared queries

Both REST and GraphQL read through the same store, so a filter or a lookup behaves identically on either surface. The list query supports the two filters both APIs expose.

Listing DS.1 - list with filters — provider/app/store.py

```
def all(self, tag: str | None = None, available: bool | None = None) -> list[Book]:
    out = list(self._books.values())
    if tag is not None:
        out = [b for b in out if tag in b.tags]
    if available is not None:
        out = [b for b in out if b.available is available]
    return sorted(out, key=lambda b: b.id)
```

Listing DS.2 - creation — provider/app/store.py

```
def add(self, title: str, author: str, isbn: str, year: int,
        tags: list[str] | None = None) -> Book:
    b = Book(self._next, title, author, isbn, year, tags or [],
             True, f"https://covers.accord.test/{self._next}.jpg", 0.0)
    self._books[b.id] = b
    self._next += 1
    return b
```

3.2 Lookup by author

The by-author query underlies the REST /authors/{author}/books endpoint and is a plain filter over the same store -- small, but a real query the provider tests pin (two books for Octavia E. Butler).

Listing DS.3 - lookup by author — provider/app/store.py

```
def by_author(self, author: str) -> list[Book]:
    return [b for b in self.all() if b.author == author]
```

CHAPTER 4

The REST surface

> *Thin routes and one mapping function -- the single place the response contract lives.*

4.1 The response shape

Every REST response is built by one function that maps a Book to the declared shape. Concentrating the contract here means a breaking change (a renamed or retyped field) is a one-line edit -- which is exactly how the catalog expresses several of them.

Listing 3.1 - the REST response mapping — provider/app/rest.py

```
def _to_out(b: Book) -> dict:
    return {
        "id": b.id,
        "title": b.title,
        "author": b.author,
        "isbn": b.isbn,
        "year": b.year,
        "tags": b.tags,
        "available": b.available,
        "cover_url": b.cover_url,
        "rating": b.rating,
    }
```

Listing 3.2 - a route (status codes are part of the contract) — provider/app/rest.py

```
@r.get("/books/{book_id}", response_model=BookOut)
def get_book(book_id: int):
    b = store.get(book_id)
    if b is None:
        raise HTTPException(status_code=404, detail="book not found")
    return _to_out(b)
```

CHAPTER 5

The rest of the REST

> *List, create, and the by-author endpoint -- and the models that declare the request and response shapes.*

5.1 The declared shapes

The request and response shapes are Pydantic models. The response model is the provider's promise; several catalog changes edit exactly this file.

Listing RR.1 - the response model — provider/app/models.py

```
class BookOut(BaseModel):
    id: int
    title: str
    author: str
    isbn: str
    year: int
    tags: list[str]
    available: bool
    cover_url: str
    rating: float
```

Listing RR.2 - the create request model (input validation) — provider/app/models.py

```
class BookCreate(BaseModel):
    title: str = Field(min_length=1, max_length=300)
    author: str = Field(min_length=1, max_length=200)
    isbn: str = Field(min_length=10, max_length=20)
    year: int = Field(ge=1400, le=2100)
    tags: list[str] = Field(default_factory=list)
```

5.2 List, create, by author

Listing RR.3 - list with query filters — provider/app/rest.py

```
@r.get("/books", response_model=list[BookOut])
def list_books(tag: str | None = Query(default=None),
               available: bool | None = Query(default=None)):
    return [_to_out(b) for b in store.all(tag=tag, available=available)]
```

Listing RR.4 - create (201 Created) provider/app/rest.py

```
@r.post("/books", response_model=BookOut, status_code=201)
def create_book(body: BookCreate):
    b = store.add(body.title, body.author, body.isbn, body.year, body.tags)
    return _to_out(b)
```

5.3 The by-author endpoint

The by-author route returns a 404 when the author has no books, so the 'empty' and 'unknown' cases are one status -- a small information-leak guard, and a route the provider tests cover on both paths.

Listing RR.5 - books by author provider/app/rest.py

```
@r.get("/authors/{author}/books", response_model=list[BookOut])
def books_by_author(author: str):
    books = store.by_author(author)
    if not books:
        raise HTTPException(status_code=404, detail="author has no books")
    return [_to_out(b) for b in books]
```

CHAPTER 6

Status codes as a contract

> *The HTTP status is a machine-readable promise; the tests and the schema-diff both assert it.*

6.1 The vocabulary

Table ST.1 - the status contract

Code	Meaning in Accord
200 OK	a successful read
201 Created	a book was created
404 Not Found	unknown book or author
422 Unprocessable	invalid create payload

A status code is part of the contract, so a change from 201 to 200 on create is a breaking change -- caught by the provider test that asserts 201 and by the schema-diff's status probe, though not by the consumer contract, which never creates a book. That split is recorded in the matrix.

CHAPTER 7

Testing the unhappy paths

> *The error contract -- 404s and 422s -- is as much a promise as the happy path.*

7.1 Errors are part of the API

A client branches on 404 and 422 as surely as on 200, so those paths are tested explicitly. An endpoint that starts returning 200 with an empty body instead of 404 is a breaking change even though nothing 'errored'.

Listing ER.1 - a missing book is 404 — testing/provider_tests/test_rest.py

```
def test_get_missing_is_404(client):
    assert client.get("/api/books/999").status_code == 404
```

Listing ER.2 - an author with no books is 404 — testing/provider_tests/test_rest.py

```
def test_author_without_books_is_404(client):
    assert client.get("/api/authors/Nobody/books").status_code == 404
```

Listing ER.3 - GraphQL filtering, pinned — testing/provider_tests/test_graphql.py

```
def test_books_filter_by_tag(client):
    r = gql(client, '{ books(tag: "classic") { id title } }')
    ids = {b["id"] for b in r["data"]["books"]}
    assert ids == {1, 3, 5}
```

CHAPTER 8

The GraphQL surface

> *The same data, a typed schema -- and why that typing is a testing asset.*

8.1 The schema

The GraphQL type mirrors the REST shape, but it is *typed and printable*. Strawberry converts snake_case to camelCase, so cover_url becomes coverUrl in the schema.

Listing 4.1 - the GraphQL Book type — provider/app/graphql_api.py

```
class Book:
    id: int
    title: str
    author: str
    isbn: str
    year: int
    tags: list[str]
    available: bool
    cover_url: str
    rating: float
```

Listing 4.2 - the Query resolvers — provider/app/graphql_api.py

```
def build_schema(store: Store) -> strawberry.Schema:
    @strawberry.type
    class Query:
        @strawberry.field
        def books(self, tag: str | None = None) -> list[Book]:
            return [_to_gql(b) for b in store.all(tag=tag)]

        @strawberry.field
        def book(self, id: int) -> Book | None:
            b = store.get(id)
            return _to_gql(b) if b else None

    return strawberry.Schema(query=Query)
```

Printed, the schema is the contract in one glance -- and a diff of this SDL is a breaking-change detector that needs no consumer:

Listing 4.3 - the printed GraphQL SDL

```
type Book {
  id: Int!
  title: String!
  author: String!
  isbn: String!
  year: Int!
  tags: [String!]!
  available: Boolean!
  coverUrl: String!
  rating: Float!
}

type Query {
  books(tag: String = null): [Book!]!
  book(id: Int!): Book
}
```

The mapping from a stored Book to the GraphQL type is the GraphQL counterpart of the REST `_to_out` -- and the place a GraphQL field rename or retype is injected:

Listing 4.4 - mapping a Book into the GraphQL type `provider/app/graphql_api.py`

```
def _to_gql(b) -> Book:
    return Book(id=b.id, title=b.title, author=b.author, isbn=b.isbn, year=b.year,
                tags=list(b.tags), available=b.available, cover_url=b.cover_url,
                rating=b.rating)
```

CHAPTER 9

GraphQL introspection and the type map

> *The typed schema is machine-readable, which is what makes the structural diff on GraphQL so cheap.*

9.1 Building the type map

The schema-diff turns the printed SDL into a map of type -> field -> type, which is directly comparable across versions. This is the advantage a typed API gives the tester: the contract is data.

Listing IN.1 - the GraphQL type map — scoring/schema_diff.py

```
def _graphql_types(schema) -> dict:
    sdl = print_schema(schema)
    gs = gql_build(sdl)
    out = {}
    for name, t in gs.type_map.items():
        if name.startswith("__") or not hasattr(t, "fields"):
            continue
        out[name] = {fname: str(f.type) for fname, f in t.fields.items()}
    return out
```

A provider test can even assert the schema from the inside, via an introspection query -- the same mechanism a client uses to discover the API:

Listing IN.2 - asserting the schema by introspection — testing/provider_tests/test_graphql.py

```
def test_schema_exposes_book_type(client):
    # introspection: the Book type and its title field must exist
    q = '{ __type(name: "Book") { name fields { name } } }'
    r = gql(client, q)
    t = r["data"]["__type"]
    assert t["name"] == "Book"
    names = {f["name"] for f in t["fields"]}
    assert {"id", "title", "author"} <= names
```

CHAPTER 10

GraphQL's different error model

> *A GraphQL request is 200 with an errors array, not a 4xx -- which changes what the tests assert.*

10.1 Null, and the errors array

GraphQL does not use HTTP status for domain outcomes: a missing book is a 200 whose data.book is null, and a bad query is a 200 with an errors array. The provider tests assert this explicitly, because a consumer that branches on HTTP status alone would be misled.

Listing GE.1 - a missing book is null, not 404 — testing/provider_tests/test_graphql.py

```
def test_missing_book_is_null(client):
    r = gql(client, "{ book(id: 999) { id } }")
    assert r["data"]["book"] is None
```

This is a concrete way REST and GraphQL diverge for the tester: the REST 404 test and the GraphQL null test check the same domain fact through two different contracts, and a schema-diff on the REST side watches status codes while on the GraphQL side it watches nullability. Same intent, different instruments.

CHAPTER 11

The provider's own tests

> *A thorough, realistic suite -- and the fields it does not assert, which is where the consumer gap lives.*

11.1 What the provider tests

The provider's suite is genuinely thorough on behaviour: status codes, filtering, errors, and the core identity fields of a book. What it does not do -- what almost no provider suite does -- is assert every field of every response.

Listing 5.1 - the provider asserts core fields, not cover_url or rating — testing/provider_tests/test_rest.py

```
def test_get_one_core_fields(client):
    b = client.get("/api/books/1").json()
    assert b["id"] == 1
    assert b["title"] == "The Left Hand of Darkness"
    assert b["author"] == "Ursula K. Le Guin"
    assert b["year"] == 1969
    assert b["available"] is True
```

Listing 5.2 - the GraphQL provider test — testing/provider_tests/test_graphql.py

```
def test_book_by_id(client):
    r = gql(client, "{ book(id: 1) { id title author year } }")
    assert r.get("errors") is None
    b = r["data"]["book"]
    assert b["id"] == 1
    assert b["title"] == "The Left Hand of Darkness"
    assert b["author"] == "Ursula K. Le Guin"
    assert b["year"] == 1969
```

This is not a weak suite -- it is a realistic one. The gap between "what the provider asserts" and "what a consumer depends on" is universal in any multi-team system, and it is exactly the gap the consumer contract is built to close.

11.2 Behaviour, thoroughly

The provider suite is thorough where it counts -- filtering and creation are pinned end to end, so a change to that behaviour is caught here, not left to a consumer to discover.

Listing 5.3 - filtering is pinned — testing/provider_tests/test_rest.py

```
def test_filter_by_tag(client):
    r = client.get("/api/books", params={"tag": "fantasy"})
    assert r.status_code == 200
    titles = {b["title"] for b in r.json()}
    assert titles == {"A Wizard of Earthsea"}
```

Listing 5.4 - creation is pinned — testing/provider_tests/test_rest.py

```
def test_create_returns_201_and_persists(client):
    r = client.post("/api/books", json={
        "title": "New Book", "author": "A. Writer",
        "isbn": "978-0000000000", "year": 2020, "tags": ["new"]})
    assert r.status_code == 201
    new_id = r.json()["id"]
    assert client.get(f"/api/books/{new_id}").status_code == 200
```

11.3 Listing the GraphQL catalog

A last provider GraphQL test pins the full-list query -- five books, no filter -- the mirror of the REST list test.

Listing 5.5 - the GraphQL list query — testing/provider_tests/test_graphql.py

```
def test_books_list_all(client):
    r = gql(client, "{ books { id } }")
    assert len(r["data"]["books"]) == 5
```

CHAPTER 12

The seed, and testing against known data

> *A fixed catalog is what lets a test assert exact values, not just shapes.*

12.1 Known data, exact assertions

Because the store is seeded with a fixed set of books, tests assert exact facts -- five books, a specific title for id 1, two books for an author -- rather than vague shape checks. Exact assertions are sharper sensors: they fail on a data-mapping bug a shape check would wave through.

Listing SD.1 - an exact-count assertion — testing/provider_tests/test_rest.py

```
def test_list_returns_all(client):
    r = client.get("/api/books")
    assert r.status_code == 200
    assert len(r.json()) == 5
```

Listing SD.2 - the health probe — provider/app/rest.py

```
@r.get("/health")
def health():
    return {"status": "ok"}
```

CHAPTER 13

The consumer and its contract

> *A real client that renders specific fields, and the Pact-style contract that pins them.*

13.1 The consumer

The reading-list widget renders a shelf and a detail page. It depends on cover_url and rating -- the very fields the provider treats as incidental. This is the code the contract protects.

Listing 6.1 - the consumer renders cover_url — consumer/client.py

```
def shelf(self) -> list[ShelfItem]:
    """The main view: every book, rendered as a shelf card."""
    rows = self.http.get("/api/books").json()
    return [ShelfItem(r["id"], r["title"], r["author"],
                      r["cover_url"], r["available"]) for r in rows]
```

13.2 The contract

The contract is a Pact-style list of interactions: for each request the consumer makes, the response fields and types it needs -- and, for rating, a *value* matcher, because a rating is 0-5, not merely 'a float'.

Listing 6.2 - a contract interaction with a value matcher — consumer/contracts/reading-list.json

```
"description": "REST: open a book's detail",
"kind": "rest",
"request": { "method": "GET", "path": "/api/books/1" },
"response": {
  "status": 200,
  "shape": "object",
  "fields": { "id": "int", "title": "str", "author": "str", "cover_url": "str", "rating": "float" },
  "value_rules": [{ "field": "rating", "min": 0.0, "max": 5.0 }]
```

13.3 The GraphQL side of the contract

The contract covers GraphQL interactions too -- here the shelf-covers query the widget uses, depending on coverUrl (the camelCase of cover_url):

Listing 6.3 - a GraphQL contract interaction — consumer/contracts/reading-list.json

```
"description": "GraphQL: shelf covers",
"kind": "graphql",
"request": { "query": "{ books { id title coverUrl } }" },
"response": {
  "data_path": "books",
  "shape": "array",
  "fields": { "id": "int", "title": "str", "coverUrl": "str" }
```

CHAPTER 14

The consumer, over GraphQL

> *The same widget, the other surface -- and the same contract idea.*

14.1 One client, two protocols

The widget can fetch a book's detail over GraphQL as easily as over REST, asking for exactly the fields it needs -- which is GraphQL's selling point and, for the tester, the reason the contract lists the query verbatim.

Listing CL.1 - the consumer's GraphQL call — consumer/client.py

```
def detail_via_graphql(self, book_id: int) -> dict:
    q = "{ book(id: %d) { id title author rating } }" % book_id
    data = self.http.post("/graphql", json={"query": q}).json()["data"]["book"]
    return data
```

Listing CL.2 - verifying every contract — scoring/verify_contract.py

```
def verify_all(client) -> list[str]:
    out = []
    for c in load_contracts():
        out += verify(client, c)
    return out
```

CHAPTER 15

Consumer-driven contract verification

> *Replay what the consumer declared against the provider -- a dependency-free Pact.*

15.1 Verifying the provider against the consumer's needs

Verification replays every declared interaction against the provider and checks status, field presence, field type, and value matchers. Unlike the provider's own tests, it encodes the *consumer's* needs -- so it catches a break the provider never thought to test, and (by design) it is blind to changes in fields no consumer depends on.

Listing 7.1 - field, type and value checks — scoring/verify_contract.py

```
def _check_fields(elem, fields, desc, violations, rules=()) -> None:
    if elem is None:
        violations.append(f"{desc}: null where an object was expected")
        return
    for f, t in fields.items():
        if f not in elem:
            violations.append(f"{desc}: missing field '{f}'")
        elif not _type_ok(elem[f], t):
            violations.append(f"{desc}: field '{f}' is not {t}")
    # value matchers (a real contract asserts more than types: a rating is 0-5,
    # not merely 'a float' -- which is what catches a silent semantic rescale)
    for rule in rules:
        f = rule["field"]
        v = elem.get(f)
        if isinstance(v, (int, float)) and not isinstance(v, bool):
            if "min" in rule and v < rule["min"]:
                violations.append(f"{desc}: field '{f}'={v} below min {rule['min']}")
            if "max" in rule and v > rule["max"]:
                violations.append(f"{desc}: field '{f}'={v} above max {rule['max']}")
```

Listing 7.2 - replaying REST and GraphQL interactions scoring/verify_contract.py

```
def verify(client, contract: dict) -> list[str]:
    """Return a list of (deduplicated) contract violations; empty means satisfied."""
    violations: list[str] = []
    for it in contract["interactions"]:
        desc = it["description"]
        resp = it["response"]
        if it["kind"] == "rest":
            req = it["request"]
            r = client.get(req["path"]) if req["method"] == "GET" else \
                client.request(req["method"], req["path"])
            if r.status_code != resp["status"]:
                violations.append(f"{desc}: status {r.status_code} != {resp['status']}")
                continue
            body = r.json()
            elems = body if resp.get("shape") == "array" else [body]
            for el in elems:
                _check_fields(el, resp["fields"], desc, violations, resp.get("value_rules", ()))
        else:  # graphql
            env = client.post("/graphql", json={"query": it["request"]["query"]}).json()
            if env.get("errors"):
                violations.append(f"{desc}: graphql errors {env['errors']}")
                continue
            data = (env.get("data") or {}).get(resp["data_path"])
            elems = data if resp.get("shape") == "array" else [data]
            if elems is None:
                violations.append(f"{desc}: null result")
                continue
            for el in elems:
                _check_fields(el, resp["fields"], desc, violations, resp.get("value_rules", ()))
    # dedup while preserving order (a renamed field hits every array element)
    return list(dict.fromkeys(violations))
```

CHAPTER 16

Matching types across two protocols

> *One type-checker serves REST JSON and GraphQL results alike -- and why bool-is-not-int matters.*

16.1 One matcher, both surfaces

The contract verifier checks the same declared types whether the value came from REST JSON or a GraphQL result. The one subtlety worth its own line: in Python a bool is an int, so an honest 'is this an int' check must exclude booleans -- otherwise available=true would satisfy a field declared int.

Listing TY.1 - type matching, bool excluded from int — scoring/verify_contract.py

```
def _type_ok(value, tname: str) -> bool:
    if value is None:
        return False
    if tname == "int":
        return isinstance(value, int) and not isinstance(value, bool)
    if tname == "bool":
        return isinstance(value, bool)
    if tname == "float":
        return isinstance(value, (int, float)) and not isinstance(value, bool)
    if tname == "str":
        return isinstance(value, str)
    return True
```

CHAPTER 17

The schema diff

> *A structural API linter -- breaking changes with no consumer, and its one blind spot.*

17.1 Derive a descriptor, diff it

The schema diff derives a comparable descriptor of the provider -- REST paths, the shape of a book, probe status codes, and the GraphQL type map -- and compares it against a baseline from the known-good provider. It reports only *breaking* diffs and ignores additive ones, so a new field is never a false alarm.

Listing 8.1 - the provider descriptor — scoring/schema_diff.py

```
def descriptor(client, schema) -> dict:
    """A comparable snapshot of the provider's public surface."""
    app = client.app
    paths = sorted(app.openapi().get("paths", {}).keys())

    book = client.get("/api/books/1")
    book_fields = {k: _typename(v) for k, v in book.json().items()} \
        if book.status_code == 200 else {}

    probes = {
        "GET /api/books/1": client.get("/api/books/1").status_code,
        "GET /api/books/999": client.get("/api/books/999").status_code,
        "POST /api/books": client.post("/api/books", json={
            "title": "x", "author": "y", "isbn": "978-0000000000",
            "year": 2000, "tags": []}).status_code,
    }
    return {
        "rest": {"paths": paths, "book_fields": book_fields, "probes": probes},
        "graphql": _graphql_types(schema),
```

Listing 8.2 - breaking-change detection scoring/schema_diff.py

```python
def diff(base: dict, cur: dict) -> list[str]:
    """Breaking changes from `base` to `cur`. Additive changes are not breaking."""
    breaks: list[str] = []

    # REST paths removed
    for p in base["rest"]["paths"]:
        if p not in cur["rest"]["paths"]:
            breaks.append(f"REST path removed: {p}")

    # REST book fields removed or retyped
    bf, cf = base["rest"]["book_fields"], cur["rest"]["book_fields"]
    for f, t in bf.items():
        if f not in cf:
            breaks.append(f"REST field removed: book.{f}")
        elif cf[f] != t:
            breaks.append(f"REST field retyped: book.{f} {t} -> {cf[f]}")

    # REST probe status codes changed
    for probe, code in base["rest"]["probes"].items():
        if cur["rest"]["probes"].get(probe) != code:
            breaks.append(f"REST status changed: {probe} {code} -> {cur['rest']['probes'].get(probe)}")

    # GraphQL fields removed or retyped
    for tname, fields in base["graphql"].items():
        cur_fields = cur["graphql"].get(tname)
        if cur_fields is None:
            breaks.append(f"GraphQL type removed: {tname}")
            continue
        for f, t in fields.items():
            if f not in cur_fields:
                breaks.append(f"GraphQL field removed: {tname}.{f}")
            elif cur_fields[f] != t:
                breaks.append(f"GraphQL field retyped: {tname}.{f} {t} -> {cur_fields[f]}")

    return breaks
```

Its blind spot is semantics: a field whose name and type are unchanged but whose *meaning* changed looks identical. That is the one class the value matcher in the contract can catch and the diff cannot.

CHAPTER 18

The change catalog

> *Realistic API changes, breaking and additive, each an exact edit.*

18.1 A change is one or more edits

Each catalog entry is a class, a breaking flag, and a set of exact source edits. Breaking changes should be caught by at least one mechanism; additive ones by none (a mechanism flagging an additive change is a false positive, measured too).

Listing 9.1 - the Change record — scoring/catalog.py

```
class Change:
    id: str
    klass: str
    breaking: bool
    edits: tuple[tuple[str, str, str], ...]     # (file, find, replace)
    desc: str
```

Listing 9.2 - a field rename (two edits: model + mapping) — scoring/catalog.py

```
Change("C01", "rename", True, (
    (MO, "cover_url: str", "image: str"),
    (RE, '"cover_url": b.cover_url,', '"image": b.cover_url,'),
), "REST: rename cover_url -> image (a field the consumer renders)."),
```

Listing 9.3 - a semantic change: shape identical, meaning changed — scoring/catalog.py

```
Change("C12", "semantic", True, (
    (RE, '"rating": b.rating,', '"rating": b.rating * 20,'),
), "REST: rating silently rescaled 0-5 -> 0-100 (still a float)."),
```

18.2 Two more entries

A field removal and a nullability widening, each a precise edit:

Listing 9.4 - remove a field — scoring/catalog.py

```
Change("C06", "remove-field", True, (
    (MO, "    year: int\n    tags: list[str]", "    tags: list[str]"),
), "REST: drop the year field from the book response."),
```

Listing 9.5 - widen to nullable scoring/catalog.py

```
Change("C15", "nullability", True, (
    (GQ, "    title: str\n", "    title: str | None\n"),
), "GraphQL: title becomes nullable (String! -> String)."),
```

18.3 The type-change entries

The type-change class edits both the model/field and the mapping so the change is real end to end -- a bool becomes a string, an Int becomes a String:

Listing 9.6 - REST bool -> string scoring/catalog.py

```
Change("C03", "type-change", True, (
    (MO, "available: bool", "available: str"),
    (RE, '"available": b.available,', '"available": str(b.available).lower(),'),
), "REST: available becomes a string ('true') instead of a bool."),
```

Listing 9.7 - GraphQL Int -> String scoring/catalog.py

```
Change("C16", "type-change", True, (
    (GQ, "    year: int\n", "    year: str\n"),
    (GQ, "year=b.year,", "year=str(b.year),"),
), "GraphQL: year becomes String instead of Int."),
```

CHAPTER 19

The scoring harness

> *Copy, patch, run three mechanisms, record. Deterministic.*

19.1 Apply and measure

For each change the harness stages a throwaway copy, applies the edits (refusing an ambiguous marker), and runs all three mechanisms against the patched copy.

Listing 10.1 - applying a change — scoring/score.py

```
def _apply(dst: Path, change) -> None:
    for rel, find, replace in change.edits:
        target = dst / rel
        text = target.read_text(encoding="utf-8")
        n = text.count(find)
        if n != 1:
            raise RuntimeError(f"{change.id}: marker {find!r} occurs {n}x in {rel} (need 1)")
        target.write_text(text.replace(find, replace), encoding="utf-8")
```

Listing 10.2 - the three mechanisms — scoring/score.py

```
def _mechanisms(cwd: Path, baseline: str) -> dict:
    return {
        "provider_tests": _pytest_caught(cwd),
        "contract": _probe(cwd, "contract"),
        "schema_diff": _probe(cwd, "schemadiff", baseline),
```

The contract and schema-diff run in a subprocess against the patched provider, so they import the changed code, not the original. The clean provider must pass all three first, or a mechanism that always fails would look omniscient.

CHAPTER 20

Running a mechanism against patched code

> *Why the contract and schema-diff run in a subprocess.*

20.1 A fresh interpreter per probe

The contract and schema-diff must import the *patched* provider, not the original the harness already loaded. So they run in a subprocess whose working directory is the patched copy; the probe imports the app from there and exits 0 (not flagged) or 1 (flagged).

Listing PB.1 - the schema-diff probe — scoring/probe.py

```
if mode == "schemadiff":
    import schema_diff
    base = json.loads(Path(sys.argv[2]).read_text(encoding="utf-8"))
    app = create_app()
    cur = schema_diff.descriptor(TestClient(app), app.state.schema)
    breaks = schema_diff.diff(base, cur)
    print(json.dumps({"breaks": breaks}))
    return 1 if breaks else 0
```

Listing PB.2 - launching a probe — scoring/score.py

```
def _probe(cwd: Path, *args: str) -> bool:
    p = subprocess.run([PY, "scoring/probe.py", *args], cwd=cwd, capture_output=True, text=True)
    if p.returncode not in (0, 1):
        raise RuntimeError(f"probe {args} exited {p.returncode}:\n{p.stdout}\n{p.stderr[-1500:]}")
    return p.returncode == 1
```

CHAPTER 21

Turning the matrix into the finding

> *The aggregation that computes who catches what the others miss.*

21.1 Complementarity, computed

The headline numbers -- contract catches what provider tests miss, schema-diff catches what the contract can't -- are not asserted; they are computed from the matrix by simple set logic, so they cannot disagree with the per-change table.

Listing AG.1 - who catches what the others miss — scoring/score.py

```
contract_only_vs_provider = [c for c in breaking
                             if caught_by(c, "contract") and not caught_by(c, "provider_tests")]
schema_catches_contract_blind = [c for c in breaking
                                 if caught_by(c, "schema_diff") and not caught_by(c, "contract")]
contract_catches_schema_blind = [c for c in breaking
                                 if caught_by(c, "contract") and not caught_by(c, "schema_diff")]
```

CHAPTER 22

Results

> *The full matrix, who catches what the others miss, and the gaps.*

22.1 The catch matrix

Table 11.1 - every change, every mechanism (reports/catch_matrix.json)

Change	Class	Kind	provider	contract	schema	note
C01	rename	break	·	✓	✓	
C02	rename	break	·	✓	✓	
C03	type-change	break	✓	✓	✓	
C04	type-change	break	·	✓	✓	
C05	type-change	break	·	✓	✓	
C06	remove-field	break	✓	·	✓	
C07	remove-endpoint	break	✓	✓	✓	
C08	status-code	break	✓	·	✓	
C09	request-validation	break	·	·	·	missed
C10	add-field	add	·	·	·	
C11	add-field	add	·	·	·	
C12	semantic	break	·	✓	·	
C14	semantic	break	·	·	·	missed
C15	nullability	break	·	·	✓	
C16	type-change	break	✓	·	✓	

22.2 Who catches what the others miss

The complementarity is the whole point:

- ['**Contract catches, provider tests miss:** C01, C02, C04, C05, C12 -- renames and retypes of fields the provider treats as incidental but the consumer renders.', '**Schema diff catches, contract is blind:** C16, C06, C08, C15 -- changes to fields/behaviour no consumer declared (a dropped field, a status code, a nullability widening).', '**Contract catches, schema diff cannot:** C12 -- a semantic rescale caught only by the value matcher.']

Table 11.2 - catch counts by change class

Class	changes	provider	contract	schema
add-field	2	0	0	0
nullability	1	0	0	1
remove-endpoint	1	1	1	1
remove-field	1	1	0	1
rename	2	0	2	2
request-validation	1	0	0	0
semantic	2	0	1	0
status-code	1	1	0	1
type-change	4	2	3	4

22.3 The gaps

2 breaking changes slip past all three mechanisms -- the finding that matters most, because a green pipeline hides it:

Table 11.3 - breaking changes no mechanism caught

Change	Class	Why all three miss it
C14	semantic	REST: isbn format changes (hyphens stripped; still a string).
C09	request-validation	REST: create accepts implausible years (validation relaxed).

And no mechanism raised a false positive on the 2 additive changes -- the schema diff correctly treats a new field as non-breaking.

CHAPTER 23

Tracing four changes

> *One caught only by the contract-vs-provider gap, one only by schema-diff, one only by the value matcher, one by nothing.*

23.1 C01 rename: contract catches, provider misses

Renaming cover_url to image edits the REST model and mapping. The provider's own tests never assert cover_url, so they pass (provider caught: ·). The consumer renders cover_url, so its contract fails (contract caught: ✓); the schema-diff sees the field vanish (schema caught: ✓). This is the archetype of consumer-driven contracts: catching a break the provider had no reason to test.

23.2 C15 nullability: only schema-diff catches

Widening GraphQL title from String! to String breaks callers that assume non-null. But the provider test asserts title's *value* (still present), so it passes (·); the contract checks the value is a string (still true), so it passes (·). Only the schema-diff, comparing String! to String, catches it (✓). A value-present check cannot see a widened type; a structural diff can.

23.3 C12 semantic: only the value matcher catches

Rescaling rating from 0-5 to 0-100 keeps the type float. The provider does not assert rating (·); the schema-diff sees float -> float and shrugs (·). Only the contract's value matcher (rating in 0-5) catches the 90.0 (✓). This is the one change the contract catches that the schema-diff cannot -- semantics live in values, not shapes.

23.4 C14 semantic format: nothing catches

Stripping the hyphens from isbn keeps it a string, and no consumer declared isbn, so all three pass (provider ·, contract ·, schema ·). It is a genuine coverage gap: a value-format break in a field no contract pins. A regex value matcher, or a second consumer that depends on isbn, would close it -- which is the honest, actionable lesson.

CHAPTER 24

Consumer tests, and the false comfort of mocks

> *Integration tests catch breaks by running both sides; the contract is the artifact that lets each side check alone.*

24.1 The consumer against the real provider

The consumer tests drive the widget against the real provider in-process. These are integration tests: they catch a breaking change, but only by running both sides together.

Listing CT.1 - a consumer integration test — testing/consumer_tests/test_reading_list.py

```
def test_detail_has_cover_and_rating(client):
    d = ReadingListClient(client).detail(1)
    assert d["cover_url"].startswith("https://")
    assert isinstance(d["rating"], float)            # the widget shows a 0-5 rating
```

The trap they warn about is the *mock*. A consumer test that mocks the provider encodes the shape the consumer *assumes*, not the shape the provider actually returns -- so it stays green while the real provider drifts away. That is the whole reason for a contract: it is the shared, verified artifact that keeps the mock honest. The consumer records what it needs once; the provider is verified against it in its own pipeline, with no mock and no running consumer.

CHAPTER 25

REST versus GraphQL, for the tester

> *The same break is cheaper to catch on the typed surface -- but GraphQL has its own hazards.*

25.1 Typed schema, cheaper diff

On GraphQL, renaming or retyping a field is a structural change to the SDL, detectable by an offline schema diff with no server call. On REST, the shape is implicit in the JSON, so the diff must derive it from live responses (or from an OpenAPI spec, if the team keeps one accurate). The typed surface hands the tester a machine-readable contract for free; the untyped one makes them reconstruct it.

That is not a blanket win for GraphQL. Its flexibility moves risk elsewhere: a client can ask for arbitrarily nested queries (a denial-of-service surface REST rarely has), field-level authorization is easy to get wrong, and a single endpoint means status-code-based monitoring is coarser. The tester's job shifts, it does not shrink -- query-depth limits and field-auth tests replace some of the status-code and routing tests REST needs.

CHAPTER 26

The performance tests GraphQL needs

> *Flexibility has a cost, and it is testable.*

26.1 N+1 and query cost

GraphQL's let-the-client-ask-for-anything model creates a performance surface REST rarely has. Two hazards a tester should pin: the **N+1 problem** -- a query for a list of books each resolving its author can fire one database call per book unless a dataloader batches them -- and **query cost** -- a deeply nested or wide query can be arbitrarily expensive. Accord's resolvers read from an in-memory store so neither bites here, but in a real deployment both are test targets: assert that resolving N books issues O(1) not O(N) backend calls, and that a query past a depth or complexity budget is rejected.

This is the other side of the REST-vs-GraphQL coin from chapter 'REST versus GraphQL, for the tester': the typed schema gives the tester a free structural contract, and in exchange the tester takes on a performance contract the single-purpose REST endpoint did not need.

CHAPTER 27

Avoiding the break in the first place

> *The measurement argues for a discipline: additive change, deprecation windows, and a contract as the gate.*

27.1 Expand, migrate, contract

Every breaking change in the catalog has a non-breaking alternative. Instead of renaming cover_url to image, add image alongside cover_url, move consumers over, and remove cover_url only once no contract still depends on it. This is the expand/migrate/contract pattern, and it is why the additive changes in the catalog are correctly flagged by nothing: adding a field is safe, removing or retyping one is not.

The contract is what makes the 'once no consumer depends on it' step checkable rather than hopeful. In a real pipeline, each consumer publishes its contract to a broker, and the provider asks 'can I deploy this change without breaking a published contract?' before it ships. The verifier here is the single-repo version of that question; Pact's broker and its can-i-deploy tool are the multi-repo version.

27.2 Deprecation is a first-class API feature

GraphQL bakes this in with an @deprecated directive on fields; REST does it with Sunset/Deprecation headers and versioned media types. The point the measurement makes is that a breaking change is a *process* failure as much as a code one: with additive change plus a deprecation window plus a contract gate, most of the 13 breaks here never reach a consumer. The tests are the safety net; the discipline is the trapeze.

CHAPTER 28

Contract-first, schema-first

> *The contract and the schema can lead the code, not trail it.*

28.1 Design the promise, then keep it

Nothing forces a contract to be written after the API. Written first, it is a design tool: the consumer and provider teams agree the interactions, and both build to them -- the provider verifies against the contract from day one, the consumer codes against it with a stub. GraphQL makes the schema-first version natural: write the SDL, generate resolvers and client types from it, and a schema diff then guards every later edit against the agreed shape.

The measurement supports this order. The changes that slip through are semantic and request-side -- exactly the things a schema cannot express and a contract must state deliberately. Deciding those up front (rating is 0-5; year is a publication year) is cheaper than discovering them from a broken consumer, and it turns two of this study's gaps into checks that would exist by construction.

CHAPTER 29

Contracts at scale

> *One verifier here; a broker, versioning, and can-i-deploy in a real many-service estate.*

29.1 The multi-repo version

Accord has one consumer and one provider in one repo, so the verifier just reads the contract off disk. A real estate has many consumers of many providers across many repos, and the coordination problem is solved by a **broker**: each consumer publishes its contract (tagged with its version and environment), the provider verifies against every published consumer contract, and before shipping it asks **can-i-deploy** -- is there any consumer, in the environment I am deploying to, whose contract this version would break?

That question is the whole value of consumer-driven contracts at scale: it converts 'will this change break someone' from a Slack message into a gate. Everything in this project -- the interactions, the provider verification, the value matchers -- is the atom the broker orchestrates. Understanding the atom is the point; the broker is operational plumbing on top of it.

CHAPTER 30

Precision: not flagging the safe changes

> *A tool that cries wolf on every additive change is as useless as one that misses breaks.*

30.1 Additive changes must pass silently

The catalog includes 2 additive changes -- adding an optional field to REST and to GraphQL. A good detector must NOT flag these, and none of the three did (0 false positives). The schema-diff earns this by only reporting removed or retyped fields, never new ones; the contract earns it by checking that required fields are present, not that no others exist.

This matters because precision is what makes a gate trustworthy. A schema-diff that flagged every new field would be turned off within a week, and then it would catch nothing. Measuring false positives alongside catches is the only way to know a tool is safe to gate on -- the same reason this portfolio measures a naive baseline's false comfort, not just a method's hits.

CHAPTER 31

Security testing for a read API

> *No auth by design -- so the security surface is input handling, mass assignment, and GraphQL's own hazards.*

31.1 What matters when the data is public

Accord is a public catalog with no authentication, so the classic auth tests do not apply -- but a security review still has real targets, and the design answers each:

- ['**Injection / malformed input** -- the fuzzer throws hostile strings and wrong types at the write path and confirms a 4xx, never a 5xx or a stack trace.', '**Mass assignment** -- the create model accepts only title/author/isbn/year/tags, so a client cannot smuggle in an id, a rating, or an availability flag. The allowlist model is the defence, and it is visible in one place (Listing RR.2).', '**GraphQL query depth / complexity** -- a single flexible endpoint invites expensive nested queries; a production deployment adds a depth/complexity limit, which is a test in its own right.', "**Information leakage** -- errors return a short detail, not an internal trace, and the same 404 covers 'unknown' and 'no books' so a probe learns nothing extra."]

The point for a tester: 'no auth' does not mean 'no security tests' -- it moves the surface to input validation and to GraphQL's particular risks, and each of those is checkable.

CHAPTER 32

Design decisions

> *Each load-bearing choice, and the alternative it beat.*

32.1 Why a curated catalog

WHY: NAMED CHANGE CLASSES OVER RANDOM MUTATION

Random schema mutation would estimate sensitivity; a curated catalog answers 'which mechanism owns which class of real change' -- rename, retype, remove, status, semantic. The results read as a table of risks and owners, which is what a team actually needs to decide what to run.

32.2 Why value matchers in the contract

WHY: A CONTRACT ASSERTS MEANING, NOT JUST SHAPE

A type-only contract is just a schema check with extra steps. The value matcher (rating in 0-5) is what lets the contract catch a semantic rescale that every structural and type check misses -- the one thing consumer-driven contracts can do that a schema diff cannot. It is also exactly how real Pact contracts work.

32.3 Why subprocess probes

WHY: IMPORT THE PATCHED CODE, NOT THE ORIGINAL

The harness has already imported the clean provider. To measure a patched one, the contract and schema-diff must run in a fresh interpreter rooted at the patched copy. A subprocess is the honest way to guarantee that; importlib reload tricks are fragile and leak state.

CHAPTER 33

The API testing pyramid

> *Where each mechanism sits, by cost and by what it can see.*

33.1 From free to expensive

The mechanisms form a pyramid of their own, ordered by cost:

1. ['**Static (schema diff / type checks)** -- no server, no consumer, runs in milliseconds; catches structural and type breaks, blind to semantics.', '**Provider unit + integration tests** -- in-process, fast; catch what the provider asserts.', "**Consumer-driven contract verification** -- in-process, fast; catches what the consumer needs, including the provider's blind spots and (with value matchers) semantics.", '**Consumer integration tests** -- run both sides; catch breaks but require the whole stack.', '**Black-box (Newman / Schemathesis)** -- a running server; the closest to production, the slowest to run.']

The lesson the matrix teaches is to spend low and broad: a schema diff and a contract are cheap and catch most breaks between them, so they belong in every pull request. The expensive black-box layer is a thin confirmation on top, not the foundation.

CHAPTER 34

The test tooling

> *pytest configuration, and the one place third-party noise is silenced.*

34.1 Configuration as code

The suite is configured in one file: which directory holds the tests, deterministic ordering, and a single suppression for Starlette's TestClient deprecation warning -- noise from a dependency, not from Accord.

Listing TL.1 - the pytest configuration — pytest.ini

```
[pytest]
# Third-party noise: Starlette's TestClient httpx deprecation.
filterwarnings =
    ignore::DeprecationWarning
addopts = -q -p no:cacheprovider
testpaths = testing
```

Keeping the config small and explicit is itself a testing value: a reviewer can see exactly what runs and what is filtered, and the filter is scoped to one dependency's message, not a blanket 'ignore all warnings' that would hide a real one.

CHAPTER 35

Input validation, and the gap it leaves

> *The create endpoint rejects bad input -- and a relaxed bound is a regression no response-side mechanism can see.*

35.1 Rejecting bad writes

The provider validates create payloads and returns 422 on bad input, and a test pins it. But note what this test can and cannot do: it asserts that a clearly-bad payload is rejected, not that every bound is exactly right.

Listing IV.1 - a bad create payload is 422 `testing/provider_tests/test_rest.py`

```
def test_create_validation_is_422(client):
    r = client.post("/api/books", json={
        "title": "", "author": "x", "isbn": "short", "year": 3000})
    assert r.status_code == 422
```

That is why the catalog's relaxed-validation change (widening the year bound) slips past all three mechanisms: it is a request-side regression, and the contract, the schema-diff and even this test -- which sends several bad fields at once -- do not isolate it. Closing it needs a property-based test of the boundary, or a data-quality assertion on what actually got stored: exactly the instrument the next project in this series is built around.

CHAPTER 36

Fuzzing the write path

> *Property-based negative testing -- and why finding nothing is the point.*

36.1 No unhandled crash

The create endpoint is fuzzed with a wide space of malformed payloads, asserting one property: the server never answers 5xx. A well-built API rejects bad input with 422, never a crash.

Listing FZ.1 - the fuzz property — scoring/fuzz.py

```
@given(p=_payload)
def _prop_no_5xx(p):
    r = client.post("/api/books", json=p)
    if r.status_code >= 500:
        _found.append(f"HTTP {r.status_code} on {json.dumps(p)[:160]}")
    assert r.status_code < 500
```

Result: **no unhandled 5xx: Pydantic validation rejects bad input with 4xx, never a crash** across 300 generated payloads.

Finding nothing is not wasted effort -- it is cheap evidence that the Pydantic validation layer holds, and a tripwire for the day it is weakened (the catalog's relaxed-validation change is exactly that day).

CHAPTER 37

Every mechanism in CI

> *Provider tests, the contract, the scoring, the fuzzer, and a Newman run -- on every push.*

37.1 The pipeline

All of it runs in CI, so a change that weakens any layer turns the build red. A second job starts the provider and runs the Postman collection with Newman -- the same requests a QA engineer would click through, automated.

Listing CI.1 - the Newman job — .github/workflows/ci.yml

```
newman:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: { python-version: "3.12" }
    - uses: actions/setup-node@v4
      with: { node-version: "20" }
    - run: pip install -r provider/requirements.txt
    - name: Start the provider
      run: |
        python -m uvicorn app.main:app --port 8000 &
        for i in $(seq 1 30); do curl -sf http://localhost:8000/api/health && break; sleep 1; done
      working-directory: provider
    - name: Newman (Postman collection)
      run: npx --yes newman run postman/accord.postman_collection.json --env-var
baseUrl=http://localhost:8000
```

CHAPTER 38

From tests to production monitoring

> *The same contract that gates a deploy can watch production.*

38.1 A contract is a synthetic check

The contract's interactions -- 'GET /api/books returns these fields with these types' -- are exactly the shape of a synthetic monitoring check. The artifact that gates a deploy in CI can run every minute against production and page when the live API stops delivering what the consumer needs. This is the natural bridge from pre-deploy testing to post-deploy observability: one description of the consumer's needs, used in both places.

It also reframes the two coverage gaps. A semantic format change that slips past the pre-deploy checks might still be caught in production by a synthetic check with a value/regex matcher, or by a consumer's own error rate rising. Testing and monitoring are the same discipline at two points in time, and the contract is the thing they share.

CHAPTER 39

Reproducibility

> *The catch matrix is verified deterministic, not merely assumed.*

39.1 Re-run and diff

The store is seeded, the catalog is a literal list, and the JSON is written with sorted keys and no timestamps, so the catch matrix is deterministic. That claim is verified the honest way -- run the scoring, snapshot the reports, run it again, and compare the files byte for byte. They are identical. The only thing deliberately kept out of the reproducible reports is wall-clock timing, which nothing in the finding depends on.

CHAPTER 40

The finding, restated

> *One table, and the boundary around it.*

40.1 Everything, in one place

Table RC.1 - the whole project, from reports/summary.json

Question	Answer (reports/summary.json)
Breaking changes in the catalog	13
Caught by the provider's own tests	5
Caught by the consumer contract	7
Caught by the schema diff	10
Caught by at least one mechanism	11
Slip past all three	2
Contract catches, provider misses	5
Schema-diff catches, contract blind	4
Contract catches, schema-diff blind	1
False positives on additive changes	0

40.2 The paragraph

Of 13 breaking changes, provider tests catch 5, consumer contracts catch 7, schema diff catches 10; 2 slip past all three. Contracts catch 5 that the provider's own tests miss. No mechanism flagged any of the 2 additive changes (0 false positives). The three mechanisms are complementary, not ranked: the provider's tests guard what it asserts, the consumer contract guards what a consumer depends on (including the provider's blind spots and, with value matchers, meaning), and the schema diff guards structure with no consumer at all. None is sufficient alone; together they still miss a silent format change and a relaxed validation, which is the honest edge -- API safety is layered, and even the layers have gaps a data-quality or property test must close.

CHAPTER 41

What this does not establish

> *The honest boundary around a curated, single-consumer study.*

41.1 A curated catalog, one consumer

The 15 changes are hand-picked to span classes cleanly, and there is one consumer with one contract. So the catch counts are illustrative of each mechanism's *reach*, not an estimate of any team's escape rate, and the contract's blind spots are the blind spots of *this* contract -- a second consumer depending on isbn would catch the format change that here slips through. What is robust is the structure: each mechanism owns a class, and semantics escape structural and type checks.

41.2 'Caught' means one check failed

A mechanism catches a change if it produced at least one failure. The study does not grade how good the failure message is, or whether it points a developer at the cause. And the schema diff is only as good as its descriptor: it compares the shape of one book and the GraphQL type map, not every nested response in the API.

CHAPTER 42

Where to read further

> *Contract testing, schema evolution, and API testing tools.*

42.1 The sources

Consumer-driven contracts come from Ian Robinson's writing and are embodied in **Pact** (pact.io) -- this project's verifier is a tiny, dependency-free version of Pact's provider verification, including value matchers. Martin Fowler's articles "ContractTest", "ConsumerDrivenContracts" and "IntegrationContractTest" are the canonical short reads. For schema evolution, the GraphQL specification's section on non-breaking changes and tools like **graphql-inspector** and **oasdiff** (OpenAPI diff) are the practical references. For the black-box layers, the Postman/Newman docs and Schemathesis (property-based API testing from an OpenAPI schema) go further than the hand-rolled fuzzer here.

CHAPTER 43

Running the Project, Step by Step

> *From a clone to the catch matrix, on Python (+ Node for Newman).*

43.1 Prerequisites

You need **Python 3.12+**; **Node 20+** only for the optional Newman run.

Toolcheck

```
python --version
node --version
```

43.2 The provider, tests, and the finding

Everything measured

```
python -m venv .venv
.venv\Scripts\activate            # Windows
source .venv/bin/activate           # macOS / Linux
pip install -r provider/requirements.txt

python -m pytest testing            # 19 tests
python scoring/verify_contract.py   # contract satisfied on clean
python scoring/score.py             # the finding -> reports/*.json
python scoring/fuzz.py              # 0 unhandled 5xx
```

43.3 Explore the API by hand

The two surfaces

```
cd provider && python -m uvicorn app.main:app --port 8000
# REST:    http://localhost:8000/api/books
# GraphQL: http://localhost:8000/graphql
```

43.4 Postman / Newman (optional)

Black-box collection

```
cd provider && python -m uvicorn app.main:app --port 8000 &
npx --yes newman run postman/accord.postman_collection.json --env-var baseUrl=http://localhost:8000
```

43.5 What you should see

`19 passed`; the contract satisfied; `score.py` prints the matrix and writes `reports/catch_matrix.json` + `summary.json` (identical on re-run); the fuzzer reports no unhandled 5xx.

CHAPTER 44

Appendix

> *Repository map and glossary.*

44.1 A - Repository map

Where everything lives

Path	Purpose
`provider/app/rest.py`	the REST surface
`provider/app/graphql_api.py`	the GraphQL schema
`consumer/client.py`	the reading-list consumer SDK
`consumer/contracts/reading-list.json`	the Pact-style contract
`testing/`	provider + consumer tests
`scoring/verify_contract.py`	contract verification
`scoring/schema_diff.py`	structural schema diff
`scoring/catalog.py`	the change catalog
`scoring/score.py`	the fault-injection harness
`scoring/fuzz.py`	property-based fuzzer
`postman/`	Postman collection (Newman/CI)
`reports/*.json`	the source of every number here

44.2 B - Glossary

Terms used in this book

Term	Meaning
Breaking change	a change that breaks an existing client
Consumer-driven contract	the consumer declares what it needs; the provider is verified against it
Provider verification	replaying a contract's interactions against the provider
Value matcher	a contract check on a value (rating 0-5), not just its type
Schema diff	detecting breaking changes by comparing API schemas
Additive change	a new field/endpoint; non-breaking
Semantic change	same shape and type, different meaning
SDL	GraphQL Schema Definition Language -- the printed schema
Newman	the command-line runner for Postman collections
Fuzzing	throwing malformed input to find crashes